SmartPlanif — Notes de présentation et analyse critique

1. SmartPlanif | Audit des algorithmes et de l’architecture

• Allocation quotidienne bus / chauffeur / receveur

• Lecture critique du code fourni, sous l’angle recherche opérationnelle

• Architecture, heuristiques, ALNS, garanties et risques

• Périmètre : audit statique ; aucune exécution .NET ou MySQL réalisée

Commentaire expert : Les observations portent sur cette version du code, et non sur une application en production. Les propositions sont distinguées des fonctions existantes.

Code examiné : README.md ; AllocateDay.cs — archive fournie [file:1].

2. Synthèse : une approche hybride pertinente

• Construction rapide, puis amélioration par ALNS et recuit simulé

• Séparation claire entre faisabilité dure et préférences souples

• Architecture modulaire et calcul en mémoire

• Limites majeures : sélection des agents, objectif approximatif, cohérence du diagnostic

Commentaire expert : Verdict : bonne base de moteur d’aide à la décision, mais pas de preuve d’optimalité ni de validation industrielle établie.

Code examiné : ConstructionStrategies.cs ; AlnsImprover.cs ; ObjectiveFunction.cs — archive fournie [file:1].

3. Quel problème est réellement résolu ?

• Les voyages et leurs horaires sont des données d’entrée

• Chaque voyage reçoit un bus et deux agents distincts

• Les ressources sont réutilisables si leurs plannings le permettent

• Ce n’est pas un calcul complet d’itinéraires ou d’horaires

Commentaire expert : Il s’agit d’une affectation multiressource avec contraintes temporelles et cumulatives. Ne pas confondre planification de ressources et conception du réseau de transport.

Code examiné : Entities.cs ; TripAllocator.cs ; AllocateDay.cs — archive fournie [file:1].

4. Formalisation proposée : variables et couverture

• x[t,b] = 1 si le bus b réalise le voyage t

• y[t,a] et z[t,a] = affectations chauffeur et receveur

• u[t] = 1 si le voyage reste non affecté

• Pour chaque t : somme x = somme y = somme z = 1 − u[t]

• Pour chaque agent a : y[t,a] + z[t,a] ≤ 1

Commentaire expert : Cette formulation est une reformulation pédagogique ; le code n’instancie pas ces variables dans un solveur mathématique. Ajouter exclusions temporelles, heures, amplitude, catégories et indisponibilités pour un modèle exact.

Code examiné : Modélisation analytique à partir de TripAllocator.cs — archive fournie [file:1].

5. Contraintes : les catégories ne sont pas interchangeables

• Verrouillées dures : bus actif, absence de chevauchement bus et agent

• Dures désactivables : repos, heures, amplitude, maintenance, kilomètres

• Dures ou souples : centre, délégation, catégorie compatible

• Préférences pures : continuité, charge faible, âge du bus

• La conformité dépend des règles effectivement activées

Commentaire expert : Toutes les règles ne proposent pas les trois modes. Les limites par défaut sont des paramètres à valider et non une preuve de conformité réglementaire.

Code examiné : ConstraintCatalog.cs — archive fournie [file:1].

6. Architecture : responsabilités et dépendances

• Web : composants Blazor en mode Interactive Server

• Application : cas d’usage et interfaces de persistance

• Domain : entités, règles, plannings, heuristiques et ALNS

• Infrastructure : MySQL / Dapper, démonstration, paramètres JSON

• Application → Domain ; Infrastructure → Application ; Web compose les services

Commentaire expert : Le schéma du README est simplifié. La dépendance logique importante est l’inversion : l’application consomme des ports, l’infrastructure les implémente ; le domaine ne dépend pas de MySQL ou de Blazor.

Code examiné : Program.cs ; Ports.cs ; fichiers .csproj — archive fournie [file:1].

7. Flux d’une allocation de bout en bout

• 1. Contrôler la date et charger la politique

• 2. Charger voyages, ressources et affectations sur J−1 à J+1

• 3. Distinguer voyages verrouillés et voyages à affecter

• 4. Construire, améliorer, compléter, puis contrôler

• 5. Enregistrer seulement si les garde-fous le permettent

Commentaire expert : Le chargement est effectué avant la boucle d’optimisation. Le calcul est placé dans Task.Run. La limite MaxSeconds concerne l’ALNS, pas l’intégralité de la requête.

Code examiné : AllocateDay.cs ; PlanningOptions.cs — archive fournie [file:1].

8. État partagé et planification en mémoire

• ScheduleBook conserve les plannings des bus et des agents

• Le même planning d’agent couvre ses deux fonctions

• Assign / Unassign modifient simultanément les occupations

• Clone isole un candidat ALNS du plan courant

• Les affectations verrouillées occupent déjà les ressources

Commentaire expert : L’état partagé évite de traiter chauffeurs et receveurs comme deux pools indépendants de personnes. Les copies sont simples à raisonner mais ont un coût à mesurer.

Code examiné : ScheduleBook.cs ; ResourceSchedule.cs ; AllocationState.cs — archive fournie [file:1].

9. Phase 0 : diagnostic de capacité

• Pic de simultanéité obtenu par balayage des événements

• Les fins passent avant les débuts en cas d’égalité

• Comparaisons : bus globaux, bus compatibles par type, agents, heures

• Un résultat suffisant n’établit pas la faisabilité globale

• Le diagnostic ne soustrait pas explicitement toutes les occupations verrouillées

Commentaire expert : Un même bus peut être compté dans plusieurs catégories compatibles. Les tests par type n’examinent pas tous les sous-ensembles de demandes qui se disputent les mêmes bus.

Code examiné : FeasibilityAnalyzer.cs — archive fournie [file:1].

10. Diagnostic : incohérences à corriger

• Le filtre du jour de repos est appliqué même si A03 est désactivée

• Les pics incluent les marges de repos même si A07 / B09 sont désactivées

• Le budget d’heures utilise MaxDailyWork même si A08 est désactivée

• Risque : annoncer une insuffisance incompatible avec la politique réelle

• Partager les règles actives entre diagnostic et allocation

Commentaire expert : Ces points sont visibles dans les expressions du diagnostic. Le résultat doit rester indicatif tant que son modèle de disponibilité n’est pas aligné sur la politique.

Code examiné : FeasibilityAnalyzer.cs ; ConstraintCatalog.cs — archive fournie [file:1].

11. Heuristique 1 : glouton chronologique

• Trier les voyages par départ puis identifiant

• Pour chaque voyage : filtrer, noter et choisir les ressources

• Conserver immédiatement l’affectation réalisable

• Atout : simplicité, référence de comparaison, rapidité potentielle

• Faiblesse : un choix local peut consommer une ressource rare

Commentaire expert : La méthode ne revient pas sur ses choix pendant la construction. Le workflow lui ajoute néanmoins la passe finale de complétion : isoler le temps de construction pour une comparaison scientifique propre.

Code examiné : ConstructionStrategies.cs ; TripAllocator.cs — archive fournie [file:1].

12. Heuristique 2 : construction par tension

• Regrouper les voyages par type de ligne

• Tension = pic de demande / max(1, bus compatibles)

• Traiter les types par tension décroissante

• Dans chaque type : traiter les voyages avec le moins d’options

• Tous les types partagent le même état de ressources

Commentaire expert : La tension représente surtout la rareté des bus, pas la totalité des tensions humaines. Le max(1, ...) évite une division par zéro mais masque la distinction entre zéro et un bus compatible dans le dénominateur.

Code examiné : ConstructionStrategies.cs — archive fournie [file:1].

13. Difficulté et sélection des candidats

• Difficulté = min(bus éligibles, options d’agents)

• Options agents = min(chauffeurs, receveurs, union des agents / 2)

• Rejeter au premier échec de règle dure

• Score souple = somme des poids × valeurs normalisées entre 0 et 1

• Classement : score + bruit éventuel, puis départage déterministe

Commentaire expert : La difficulté est une mesure de rareté, pas le nombre exact de triplets. L’ordre est calculé avant les insertions de la phase : il n’est pas réévalué après chaque choix.

Code examiné : TripAllocator.cs ; CandidateSelector.cs — archive fournie [file:1].

14. Défaut critique : la paire d’agents n’est pas optimisée

• Le code choisit d’abord le meilleur chauffeur

• Puis il retient le premier receveur différent de ce chauffeur

• Contre-exemple : A polyvalent classé premier ; B chauffeur seulement

• Le code choisit A chauffeur et ne trouve plus de receveur

• La paire B chauffeur / A receveur est pourtant réalisable

Commentaire expert : Ce cas concerne les pools mixtes, pas le cas où tous les agents sont interchangeables. Correction proposée : examiner les paires distinctes réalisables et maximiser leur score total ; ajouter un test de non-régression.

Code examiné : TripAllocator.TryAssign ; exemple analytique — archive fournie [file:1].

15. Fonction objectif effectivement codée

• Minimiser F = P × U + D + λa × σa + λb × σb

• U : voyages non affectés ; D : déficit de score souple

• σa et σb : écarts-types des heures agents et bus

• Valeurs du code : P = 100 000 ; λa = λb = 10

• Les heures d’équité couvrent tout le ScheduleBook chargé

Commentaire expert : Le terme d’équité bus est absent de la formule simplifiée du README. La fenêtre J−1 à J+1 contribue à la charge : l’objectif n’est donc pas strictement une équité du seul jour J.

Code examiné : ObjectiveFunction.cs ; AllocationPolicy.cs — archive fournie [file:1].

16. L’objectif n’est pas une hiérarchie garantie

• Le validateur vérifie P > 0, pas sa dominance sur les autres termes

• Des poids configurables peuvent renverser la priorité de couverture

• Les scores mémorisés proviennent des contextes d’insertion

• Le même plan peut recevoir des valeurs différentes selon son historique

• Proposition : comparaison lexicographique et réévaluation cohérente

Commentaire expert : L’objectif final du rapport est évalué avant la reconstruction des scores exacts. Cette reconstruction ne réinjecte pas ses valeurs dans l’état. Éviter de présenter FinalObjective comme une évaluation exacte des scores affichés.

Code examiné : ObjectiveFunction.cs ; PolicyValidator.cs ; PlanEvaluator.cs — archive fournie [file:1].

17. ALNS : explorer de grands voisinages

• Copier le plan courant

• Choisir une destruction et une réparation selon leurs poids

• Retirer une fraction des affectations puis réinsérer les voyages en attente

• Accepter ou rejeter le candidat ; conserver séparément le meilleur

• Arrêter sur itérations, durée, stagnation ou annulation

Commentaire expert : La réparation réutilise le même moteur de règles et de choix local. Elle ne garantit pas de réinsérer tous les voyages. Faisable signifie ici respect des règles dures, pas couverture intégrale.

Code examiné : AlnsImprover.cs — archive fournie [file:1].

18. Quatre destructions, trois réparations

• Destructions : aléatoire ; pires scores ; voisinage temporel ; ressource

• Le pivot temporel cible un voyage non affecté avec probabilité 0,7 lorsqu'il en existe

• Réparations : chronologique ; plus contraint ; ordre aléatoire

• Les deux dernières peuvent bruiter le classement des ressources

• Le retrait par ressource distingue le poste chauffeur du poste receveur

Commentaire expert : Ces opérateurs sont réellement présents ; aucune réparation regret-k n'est implémentée. Une suppression d'agent agrégeant ses deux postes serait une extension utile du voisinage par ressource.

Code examiné : Operators.cs — archive fournie [file:1].

19. Recuit simulé et adaptation des opérateurs

• Si $\Delta = F(\text{candidat}) - F(\text{courant}) > 0$: acceptation $\exp(-\Delta / T)$

• Refroidissement géométrique : $T \leftarrow 0{,}998 \times T$ par défaut

• Récompenses : nouveau meilleur 33 ; amélioration 9 ; accepté 3

• Poids $\leftarrow \max(0{,}1 ; (1-\rho) \times \text{poids} + \rho \times \text{récompense})$

• Destruction et réparation reçoivent la même récompense

Commentaire expert : Il s'agit d'une adaptation à chaque itération des opérateurs sélectionnés, pas d'une réestimation par segment. L'efficacité d'un couple d'opérateurs n'est pas apprise séparément.

Code examiné : AlnsImprover.cs ; AllocationPolicy.cs — archive fournie [file:1].

20. Réglages : ce qu'ils signifient réellement

• 3 000 itérations ; 20 secondes ALNS ; stagnation 1 500

• Destruction 5–15 %, mais au moins deux affectations si possible

• Température initiale 100 ; bruit 10 % ; graine 42

• P = 100 000 rend très improbable la perte temporaire d'un voyage à T = 100

• La graine seule ne garantit pas le même résultat si l'arrêt temporel intervient

Commentaire expert : Ces valeurs sont les valeurs par défaut des classes ; les paramètres JSON ou ceux de l’interface peuvent les remplacer. Pour la répétabilité, fixer données, ordre, politique et budget en itérations.

Code examiné : AllocationPolicy.cs ; AlnsImprover.cs — archive fournie [file:1].

21. Complétion et validation : garanties limitées mais utiles

• La complétion retente les voyages restants sans déplacer les affectés

• Le contrôle retire temporairement un voyage puis rejoue ses règles dures

• Il vérifie aussi les qualifications et la distinction des agents

• Il réutilise les mêmes implémentations de règles que la construction

• Dans AllocateDay, les affectations verrouillées ne sont pas toutes auditées isolément

Commentaire expert : Le contrôle est indépendant du chemin de construction, mais pas une seconde spécification indépendante. Une erreur commune à une règle échappe aux deux mécanismes. KpiService reconstruit une fenêtre de contrôle différente.

Code examiné : PlanCompleter.cs ; PlanValidator.cs ; AllocateDay.cs — archive fournie [file:1].

22. Persistance et exploitation

• Simulation sans écriture ; compléter ou remplacer le plan

• Écriture transactionnelle des seules colonnes prévues *_PR

• Protection des voyages aux trois ressources réelles renseignées

• Refus si violations ou journée incomplète dans le mode prévu

• Atomicité d’écriture ≠ protection contre un calcul devenu obsolète

Commentaire expert : La transaction débute après le calcul. Aucune vérification explicite de version de l’instantané n’apparaît dans cette méthode. Proposition : versionner le jour ou revalider sous contrôle de concurrence avant commit.

Code examiné : AllocateDay.cs ; MySqlAssignmentRepository.cs — archive fournie [file:1].

23. Limites métier et qualité des données

• Continuité de station : préférence souple, pas contrainte de trajet à vide

• Pas de matrice explicite des temps de transfert dans les règles inspectées

• Capacité contrôlée seulement si ExpectedPassengers est connu

• Durées futures parfois estimées depuis l’itinéraire de ligne

• Polyvalence par défaut et activation automatique des bus à valider

Commentaire expert : Un planning sans chevauchement n’est pas nécessairement exécutable géographiquement. Prévoir temps de transfert + préparation entre les stations, ainsi que des qualifications et indisponibilités fiables.

Code examiné : SoftRules.cs ; BusRules.cs ; README.md — archive fournie [file:1].

24. Forces et faiblesses de l’architecture

• Forces : domaine autonome, ports explicites, règles extensibles, mode démo

• Forces : calcul en mémoire, explications, progression et sauvegarde séparée

• Faiblesses : listes parcourues souvent, clones répétés et tris à chaque ajout

• Risques : paramètres JSON et calculs concurrents à gouverner

• Program.cs ne montre pas de configuration explicite authentification / autorisation

Commentaire expert : Ne pas déduire un défaut de sécurité de toute l’infrastructure déployée à partir de ce fichier seul. Pour une exploitation multiutilisateur, examiner également proxy, hébergement, identité et droits applicatifs.

Code examiné : ResourceSchedule.cs ; Program.cs ; Ports.cs ; AllocateDay.cs — archive fournie [file:1].

25. Performance et niveau de preuve

• Aucun benchmark exécuté dans cet audit

• README : 34/36 glouton ; 35/36 phases ; 36/36 ALNS ; zéro violation annoncé

• README : 58 tests annoncés avec lanceur de remplacement

• Dapper / MySqlConnector réels et interactions navigateur non validés selon le README

• Un cas de démonstration ne prouve ni supériorité générale ni passage à l’échelle

Commentaire expert : Coût d’une insertion : filtrer et noter les ressources, avec scans de leurs plannings, puis trier les candidats. Répété en réparation et en mesure de difficulté, ce coût doit être profilé plutôt que résumé par une complexité linéaire trompeuse.

Code examiné : README.md ; ResourceSchedule.cs — archive fournie [file:1].

26. Protocole scientifique recommandé

• Même jeu de données, politique, verrouillages et budget pour chaque méthode

• Petites instances : modèle exact de référence et borne de qualité

• Instances réalistes : journées ordinaires, pics, pénuries, absences, passages de minuit

• Plusieurs graines ; médiane, dispersion, pire cas et temps de calcul

• Mesurer couverture, violations, qualité exacte, équité et robustesse aux perturbations

Commentaire expert : Effectuer des ablations : phases seules, ALNS non adaptatif, sans bruit, sans chaque opérateur. Comparer la construction seule et la chaîne complète, car la passe finale modifie les résultats.

Code examiné : Recommandations issues des limites observées — archive fournie [file:1].

27. Feuille de route priorisée

• P0 : corriger la sélection de paire et aligner le diagnostic sur les règles actives

• P0 : rendre la priorité de couverture explicite ; aligner score recherché et score publié

• P0 : tester les vraies dépendances et sécuriser les écritures concurrentes

• P1 : temps de transfert, qualifications, contrôle des verrouillés, équité adaptée aux disponibilités

• P2 : regret-k, échanges ciblés, profilage, calcul incrémental, référence exacte

Commentaire expert : Critères de sortie : tests de non-régression pour chaque défaut, absence de violations sur jeux métier approuvés et protocole de benchmark reproductible. Les propositions P1/P2 ne décrivent pas des fonctionnalités existantes.

Code examiné : Synthèse de l’audit statique — archive fournie [file:1].

28. Conclusion et traçabilité

• Bon socle modulaire pour une aide à la décision explicable

• ALNS apporte une capacité de remise en cause des choix gloutons

• Le code ne fournit pas de certificat d’optimalité

• La priorité est la cohérence du modèle avant l’ajout de métaheuristiques

• Source unique : archive texte du dépôt SmartPlanif fournie par l’utilisateur

Commentaire expert : Lecture des classes citées dans chaque diapositive. Les résultats du README sont des déclarations du projet, non des mesures reproduites. Les contre-exemples et recommandations constituent l’analyse de cet audit.

Code examiné : Fichier fourni : salah1892-smartplanif-ff9b3838b8c3f414.txt — archive fournie [file:1].